Product Master — Product Type — Detailed Design Document (詳細設計書)

Document control (改版履歴)

Field	Value
Document ID	004_DD-TYPE
Screen / work item	SCR-004 「製品マスタ」 (Product master) / WI-017
Based on	004_BD-TYPE v1 (DEC-007), 004_DB-TYPE v1 (DEC-008)
Created / updated	Claude (for ThanhTN), 2026-10-25

Version	Date	Author	Revision content
1	2026-10-25	Claude (for ThanhTN)	Initial creation

This document specifies the screen side of the product type: modules, fields, states and tests. Everything it does not mention is as 004_DD and the WI-013 implementation (list layout, rows per page, return after save, unit locking, retire dialog, forbidden state).

Reference documents and companions

No.	Document	Purpose
1	004_BD-TYPE v1	Items 8–10, values, validation, events
2	004_DB-TYPE v1	Column, constraint, migration, grant
3	004_DD-API-TYPE (next review stop)	Exact request/response/query changes and errors on `/api/product-master`
4	004_DD-SPD-TYPE (review stop after that)	Filter and save sequences including `type`

Backend service changes are small enough to live in 004_DD-API-TYPE's processing section; no separate FN addendum (the FN-044/FN-045 rules are fully stated there).

Component and file organization

Path / module	Change
`ProductionManagementAI.Domain/ProductionOrders/ProductType.cs`	New enum `ProductType { Finished, SemiFinished, Part, RawMaterial }`
`ProductionManagementAI.Domain/ProductionOrders/Product.cs`	`public ProductType Type { get; set; }`
`ProductionManagementAI.Infrastructure/ProductionOrders/ProductionOrderConfigurations.cs`	Map `Type` to `product_type varchar(20)`, string converter to the DB codes, `ck_products_product_type`
`ProductionManagementAI.Infrastructure/Migrations/<ts>_AddProductType.cs`	004_DB-TYPE migration steps 1–4
`ProductionManagementAI.Application/Products/ProductMasterContracts.cs`, `ProductMasterService.cs`	`type` in list query, item, create, update; validation
`ProductionManagementAI.Infrastructure/Products/ProductMasterRepository.cs`	Filter by type
`src/frontend/src/features/products/api.ts`	`productTypes` constant, `ProductType` type, `type` in item/draft/list params
`src/frontend/src/features/products/ProductMasterPage.tsx`	Filter select, column, card line, form select, validation, focus
`src/frontend/src/features/production-orders/messages.ts`	`products` words below

No new dependency or shared component.

Screen layout and mockup

English-captioned mockup and Japanese-captioned edition, built on the 004_DD mockup styles; orange dashed outlines mark the additions. States: (1) list filtered by 「部品」 (PC), (2) create form with the type error, (3) edit of an order-referenced product — type editable, unit locked, (4) SP cards. Layout authority stays the BD-TYPE wireframes.

Field definitions and validation

Type values (shared by backend and frontend)

Code (API, DB)	C# enum	Label	Order
`FINISHED`	`Finished`	完成品	1

Code (API, DB)	C# enum	Label	Order
`SEMI_FINISHED`	`SemiFinished`	半製品	2
`PART`	`Part`	部品	3
`RAW_MATERIAL`	`RawMaterial`	原材料	4

Frontend: `export const productTypes = ['FINISHED', 'SEMI_FINISHED', 'PART', 'RAW_MATERIAL'] as const` in `api.ts`; labels in `messages.ts` `products.types` record keyed by code. Options render in this order.

Screen items

No.	Item	Element / id	Rule
8	Filter 「種別」	`<select id="type-filter" name="type">`, label 「種別」	Options: value `""` 「すべて」, then the four codes. Initial value from URL `type` if it is one of the codes, else `""`. Placed after 「状態」 in the filter grid (grid tracks keep the zero minimum from WI-014)
9	Column 「種別」	`<td>` after 製品名; SP card `<span>種別: {label}</span>` before 単位	Label from `products.types[item.type]`
10	Form 「種別」	`<select id="type" name="type" required>`, label 「種別」 + required marker	Create: initial `""` shown as 「選択してください」. Edit: loaded `item.type`. Never read-only

Client validation (on Save, before the request, like the other fields)

Condition	Field message	Focus order
`draft.type` not one of `productTypes`	MSG-E026 「種別を選択してください。」	After 製品名, before 単位 (follows the visual order)

Server `VALIDATION` on `type` (400) maps to the same message, as the page already maps server field errors.

Messages (`products` words)

Key	Text	ID
`type`	種別	—
`typePlaceholder`	選択してください	—
`types`	`{ FINISHED: '完成品', SEMI_FINISHED: '半製品', PART: '部品', RAW_MATERIAL: '原材料' }`	—

Key	Text	ID
`typeRequired`	種別を選択してください。	MSG-E026 (next free error ID after MSG-E025)

Loading, empty, error and success states

No new state. A type filter with no matches shows the existing 「条件に一致する製品はありません。」. The saved-but-hidden notice (WI-013) also fires when a saved product's type no longer matches the applied type filter.

URL state

Param	Parse	Write
`type`	Kept only if it is one of `productTypes`; otherwise removed with `replace` navigation, like an invalid `state`	Set on 検索 when not `""`; removed on クリア or when 「すべて」 is searched. Preserved by paging, 表示件数 and the return URL passed to create/edit

The list request sends `type` only when set.

Screen-owned module changes

Module	Change
`ProductDraft`	`+ type: ProductType | ''`; dirty check includes it
`toRequest(draft)`	Sends `type`
`parseListView / buildListSearch` (page-local URL helpers)	Read/write `type`
`validate(draft)`	Adds the type rule above
Focus-first-invalid	Order: sku, name, type, unit, drawingNumber

Accessibility, security and observability

- Both selects have `<label for>`; the required marker is the existing text marker; errors use `aria-invalid` and `aria-describedby="type-error"`, and appear in the error summary (WCAG 2.2 AA, same as other fields).

- The type is not sensitive; no telemetry change. Server re-validates every value (004_DD-API-TYPE).

API and persistence mapping

Screen	API (detail in 004_DD-API-TYPE)	DB
Filter 「種別」	`GET /api/product-master?type=PART`	`WHERE product_type = 'PART'`
Column / card	`items[].type`	`product_type`
Form	`POST` / `PUT` body `type` (required); `GET {id}` returns `type`	`product_type`

Test viewpoints

Level	Viewpoint
Backend unit	Validation: missing/unknown/each valid `type` on create and update; list query accepts each code, rejects others
Integration	Create/update round-trip; filter returns only that type and combines with `q` / `state` /paging; 400 on bad `type` ; migration on a populated DB gives the 004_DB-TYPE mapping and `FINISHED` for other rows; `pmai_app` can update `product_type`
Frontend unit	Filter URL parse/write (valid, invalid, クリア); column and card label; create without type → MSG-E026 and focus; edit preloads type; type editable when unit locked; dirty guard on type change
E2E	Create with type → list shows it; filter by 部品 → only parts, URL `type=PART` , kept across paging and return-after-save; edit type; axe on list and form; SP card shows type
Regression	Existing E2E/fixtures that create products send a type